

AI in signal and audio processing

Thomas de Jaeger

thomas.de-jaeger@epita.fr

Syllabus

- **Session 1:** Introduction & Fundamentals of Signal/Audio
- **Session 2:** Audio Features & Librosa
- **Session 3:** Audio Processing with Torchaudio
- **Session 4:** Sound Detection and Classification
- **Session 5:** Sequence Modeling with RNNs and LSTMs
- **Session 6:** Automatic Speech Recognition (ASR)
- **Session 7:** Speaker and Emotion Recognition
- **Session 8:** Project presentation

Sequence Modeling with RNNs and LSTMs

Session 5

Sequence Modeling with RNNs and LSTMs

Session 5

Objectives :

- Understand why sequence modeling is important for audio
- Learn the basics of Recurrent Neural Networks (RNNs)
- Identify the problems of RNNs (vanishing/exploding gradients)
- Understand how LSTMs and GRUs solve these problems
- Practice with PyTorch LSTM

Sequence Modeling with RNNs and LSTMs

Session 5

Objectives :

- Understand why sequence modeling is important for audio
- Learn the basics of Recurrent Neural Networks (RNNs)
- Identify the problems of RNNs (vanishing/exploding gradients)
- Understand how LSTMs and GRUs solve these problems
- Practice with PyTorch LSTM

Why Sequences Matter in Audio?

- **Audio is Inherently Sequential**

Audio signals have strong temporal dependencies. If you an audio into small pieces and shuffle them, the result would no longer make sense.

- Speech: words depend on previous words for context
- Music: notes create melodies, rhythms have patterns

- **Example:**

The phrase "I saw a bat" - meaning depends on what comes next

- "I saw a bat flying" (animal)
- "I saw a bat and ball" (sports equipment)

Standard neural nets (CNN) can't "remember" the past.

How do we build models that understand temporal context?

Recap - CNNs for Audio

- **What We've Done So Far : CNNs**

Input Spectrogram → Conv2D → Pool → Conv2D → Pool → Flatten → Dense → Output

- Extract local features (frequency patterns)
- Work well for classification tasks
- Use pooling to reduce dimensionality

- **Limitations:**

- CNNs can only look at a fixed window of time.
- Cannot remember information from far in the past.
- Max/average pooling loses temporal order information.
- Loss of sequence order. Same output irrespective of the order of features.
- Inability to capture long-term dependencies.

- **We need models that can:**

- Process sequences of variable length
- Maintain memory of past inputs
- Model temporal dependencies explicitly

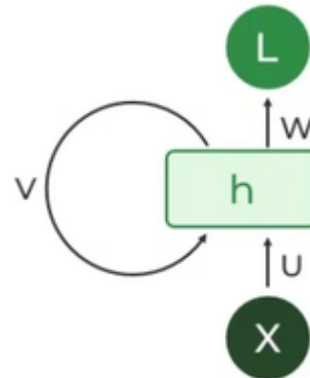
Recurrent Neural Networks (RNNs)

A recurrent neural network or RNN is a deep neural network trained on sequential or time series data to create a machine learning (ML) model that can make sequential predictions or conclusions based on sequential inputs

- While standard neural networks pass information in one direction i.e from input to output, RNNs feed information back into the network at each step.

Key Concept: At each time step, the RNN:

- Takes current input
- Combines it with previous hidden state
- Produces output and new hidden state



Recurrent Neural Networks (RNNs)

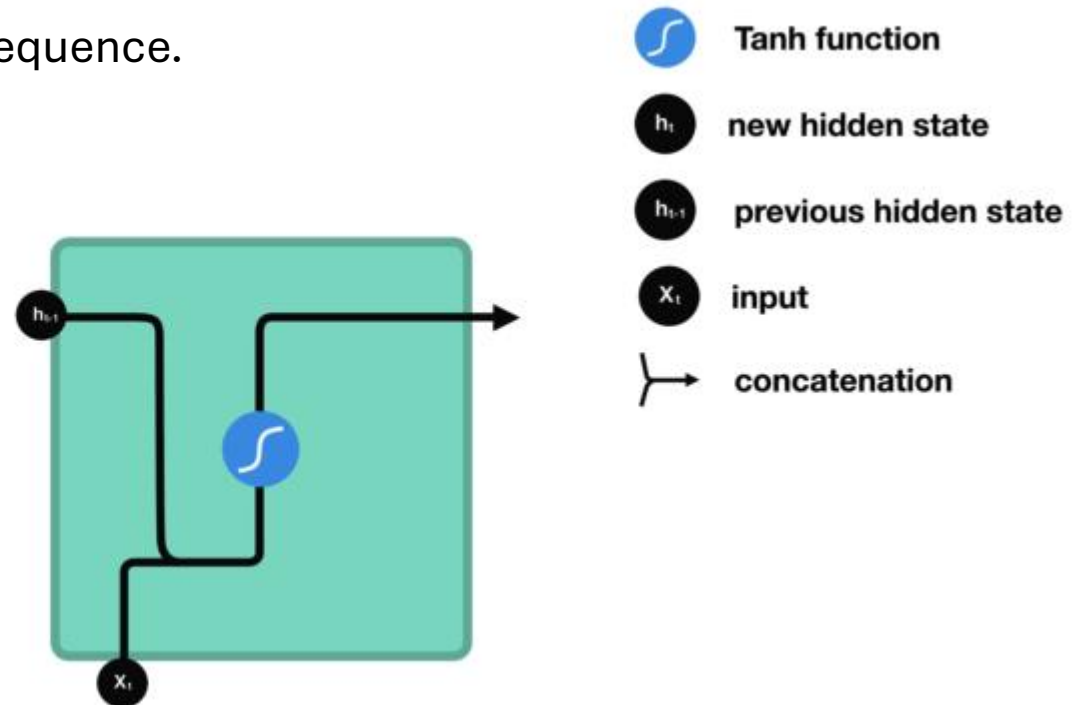
The basic structure includes:

Input Layer: Receives the sequence data.

Hidden Layer: Processes input and maintains information from earlier time steps through recurrent connections.

Output Layer: Generates predictions based on the current hidden state

At each timestep, the recurrent cell takes two inputs:
its previous hidden state and the next step of the input sequence.
These two inputs inform the next hidden state,
which creates the network's output for that step.



Recurrent Neural Networks (RNNs)

The best way to visualize this process is by ‘unrolling’ the recurrent neural network into a series of conventional feedforward neural.

This unrolling enables backpropagation through time (BPTT) a learning process where errors are propagated across time steps to adjust the network’s weights enhancing the RNN’s ability to learn dependencies within sequential data.

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

$$y_t = W_{hy}h_t + b_y$$

x_t : input at time t

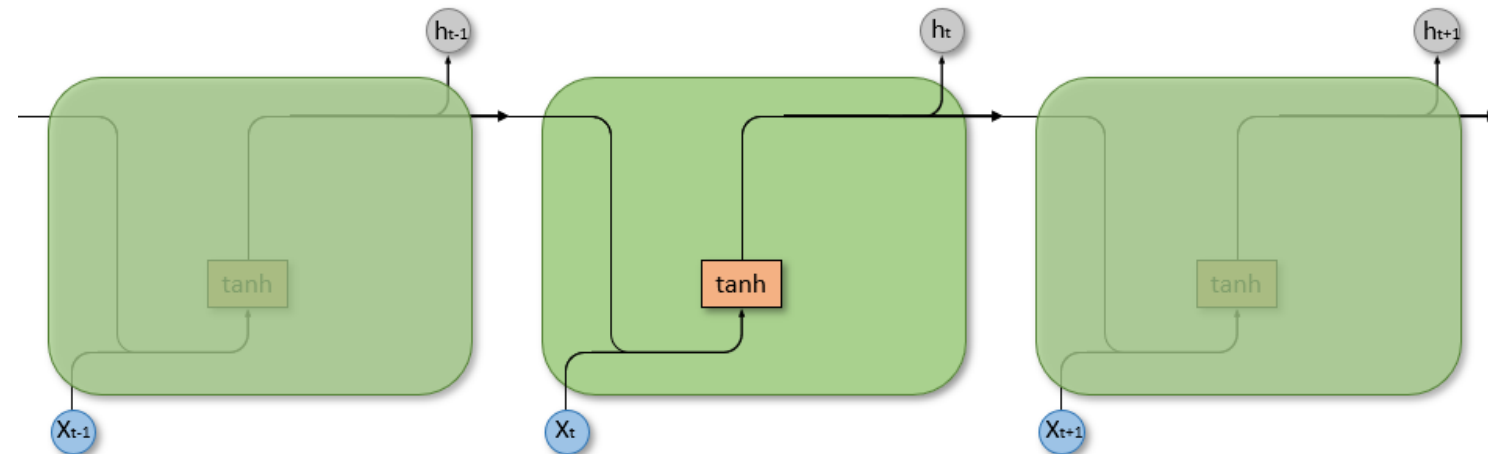
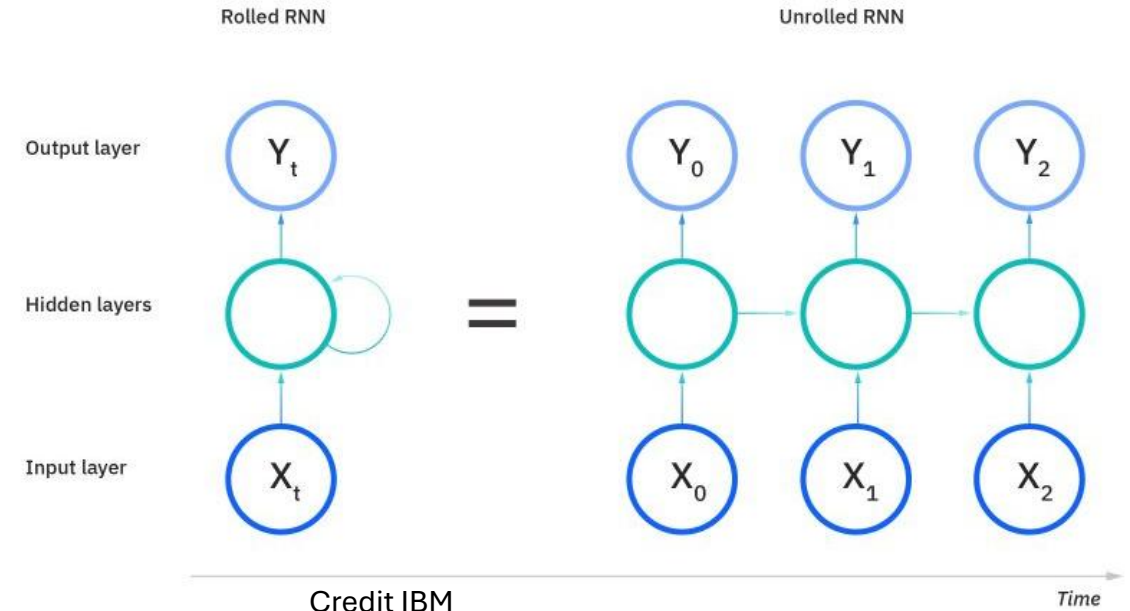
h_t : hidden state at time t

y_t : output at time t

W_{hh} : hidden-to-hidden weights

W_{xh} : input-to-hidden weights

W_{hy} : hidden-to-output weight



Recurrent Neural Networks (RNNs)

The best way to visualize this process is by ‘unrolling’ the recurrent neural network into a series of conventional feedforward neural.

These parameters are updated using backpropagation. However, since RNN works on sequential data here we use an updated backpropagation which is known as **backpropagation through time**

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

$$y_t = W_{hy}h_t + b_y$$

x_t : input at time t

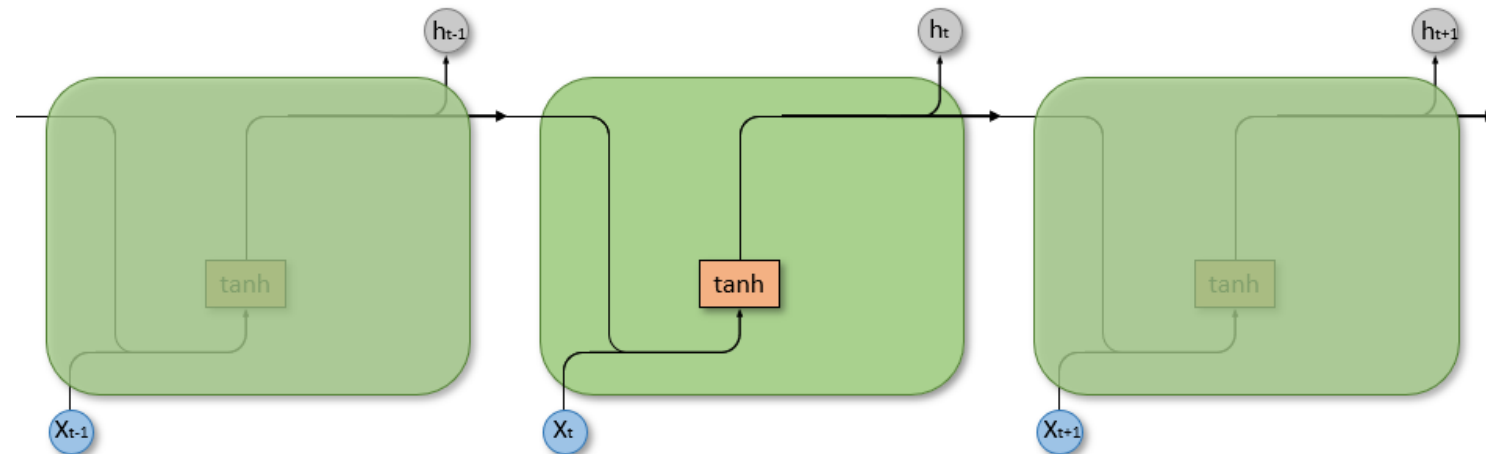
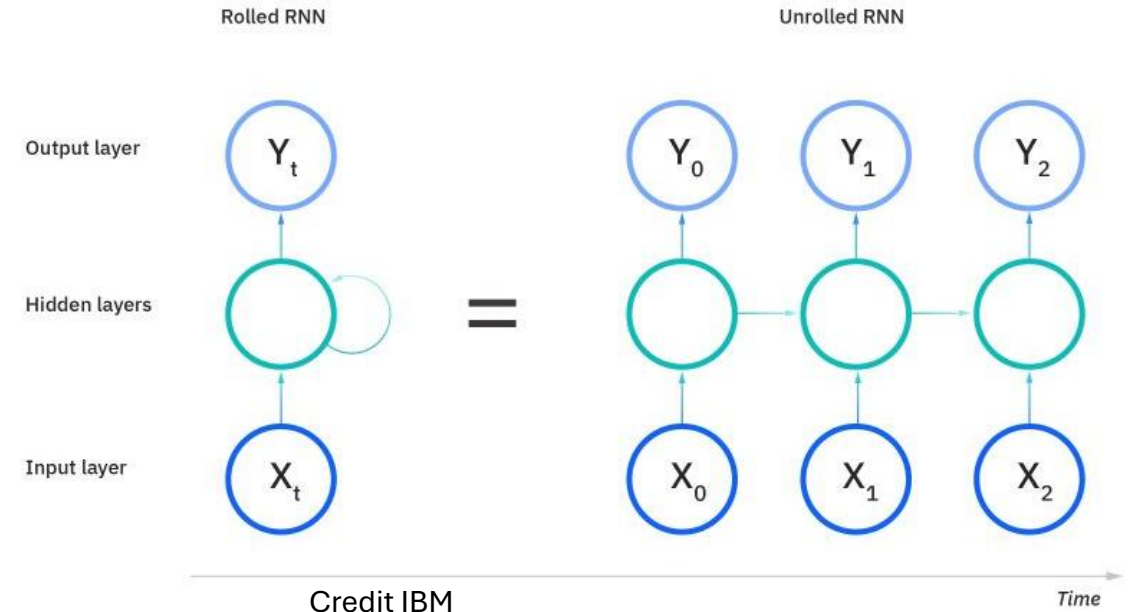
h_t : hidden state at time t

y_t : output at time t

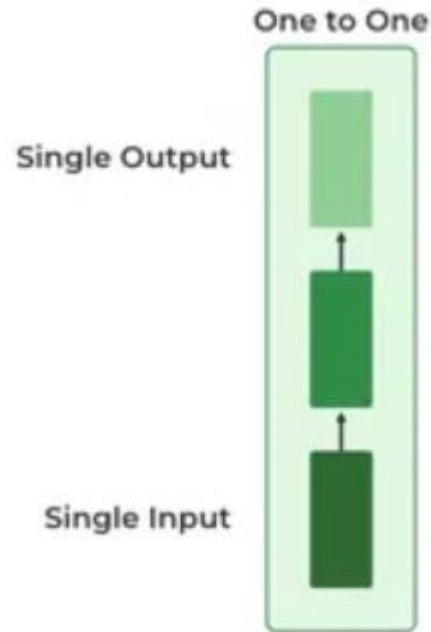
W_{hh} : hidden-to-hidden weights

W_{xh} : input-to-hidden weights

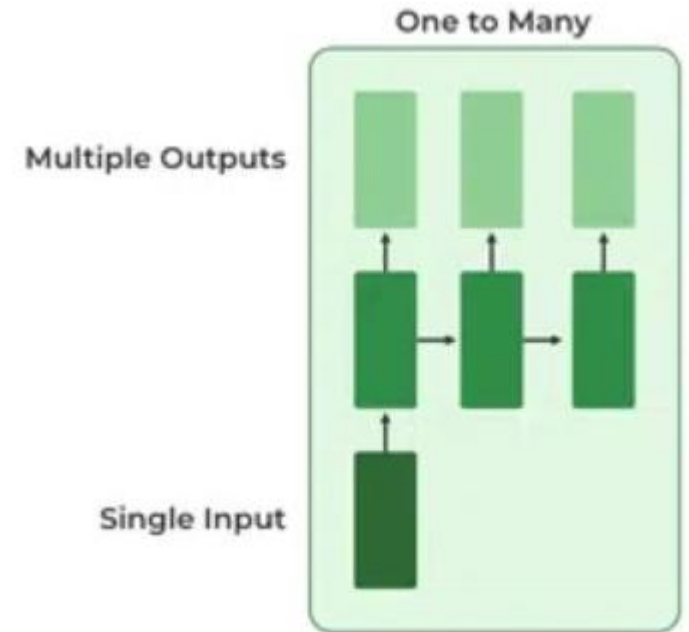
W_{hy} : hidden-to-output weight



Types Of Recurrent Neural Networks

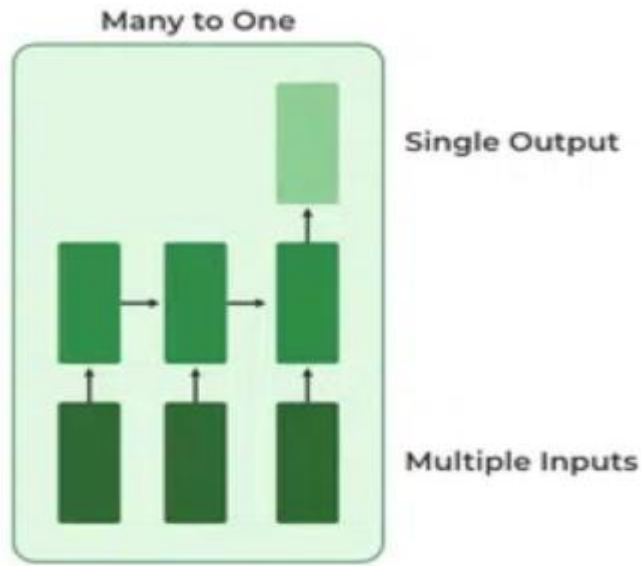


This is the simplest type of neural network architecture where there is a single input and a single output. It is used for straightforward classification tasks such as binary classification where no sequential data is involved.

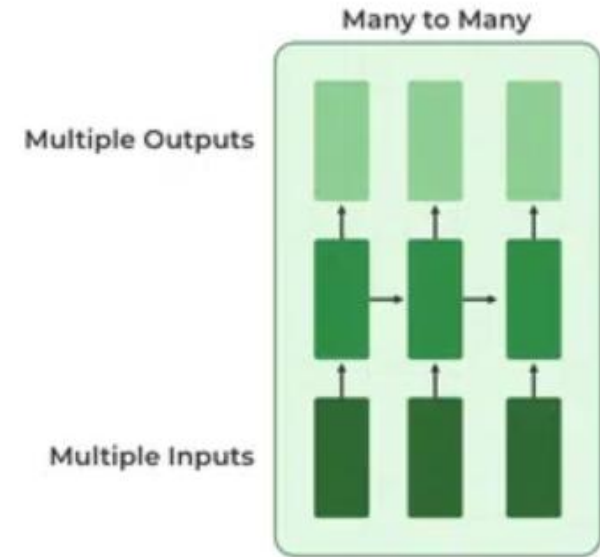


In a One-to-Many RNN the network processes a single input to produce multiple outputs over time. This is useful in tasks where one input triggers a sequence of predictions (outputs). For example in image captioning a single image can be used as input to generate a sequence of words as a caption.

Types Of Recurrent Neural Networks



The **Many-to-One RNN** receives a sequence of inputs and generates a single output. This type is useful when the overall context of the input sequence is needed to make one prediction. In sentiment analysis the model receives a sequence of words (like a sentence) and produces a single output like positive, negative or neutral.



The **Many-to-Many RNN** type processes a sequence of inputs and generates a sequence of outputs. In language translation task a sequence of words in one language is given as input and a corresponding sequence in another language is generated as output

When can we use RNN

- Speech recognition
- Music generation
- Sentiment classification
- DNA sequence analysis
- Machine translation
- Video activity recognition
- Name entity recognition
- All tasks with sequential data structures
- Image captioning
- Time series prediction (e.g. stock market trend prediction)
- Handwriting generation
- ...

LSTM: Applications

```
import torch.nn as nn
```

```
rnn = nn.RNN(input_size, hidden_size, num_layers, batch_first=True,  
nonlinearity='tanh')
```

- `input_size`: features per time step
- `hidden_size`: neurons in each LSTM layer
- `num_layers`: stacked LSTM layers

Limitations : RNN

- **Vanishing Gradient Problem:** When backpropagating over many time steps, gradients tend to shrink exponentially, making early time steps contribute very little to weight updates. This causes the network to “forget” long-term dependencies.

Ex: I was born in France, but I have been working in South Africa working for ... (another 200 words)
... Therefore my mother tongue is:

- **Exploding Gradient Problem:** Gradients may also grow uncontrollably large, causing unstable updates and making training difficult.

SOLUTION: Two specialised RNNs were created that make use of an internal mechanism called “gates” that can regulate the flow of information. Gating is a technique that helps the network learn which data in a sequence is important to keep (by adding it to the hidden state) and which should be thrown away

- Long Short-Term Memory (LSTM)
- Gated Recurrent Units (GRU)

LSTM

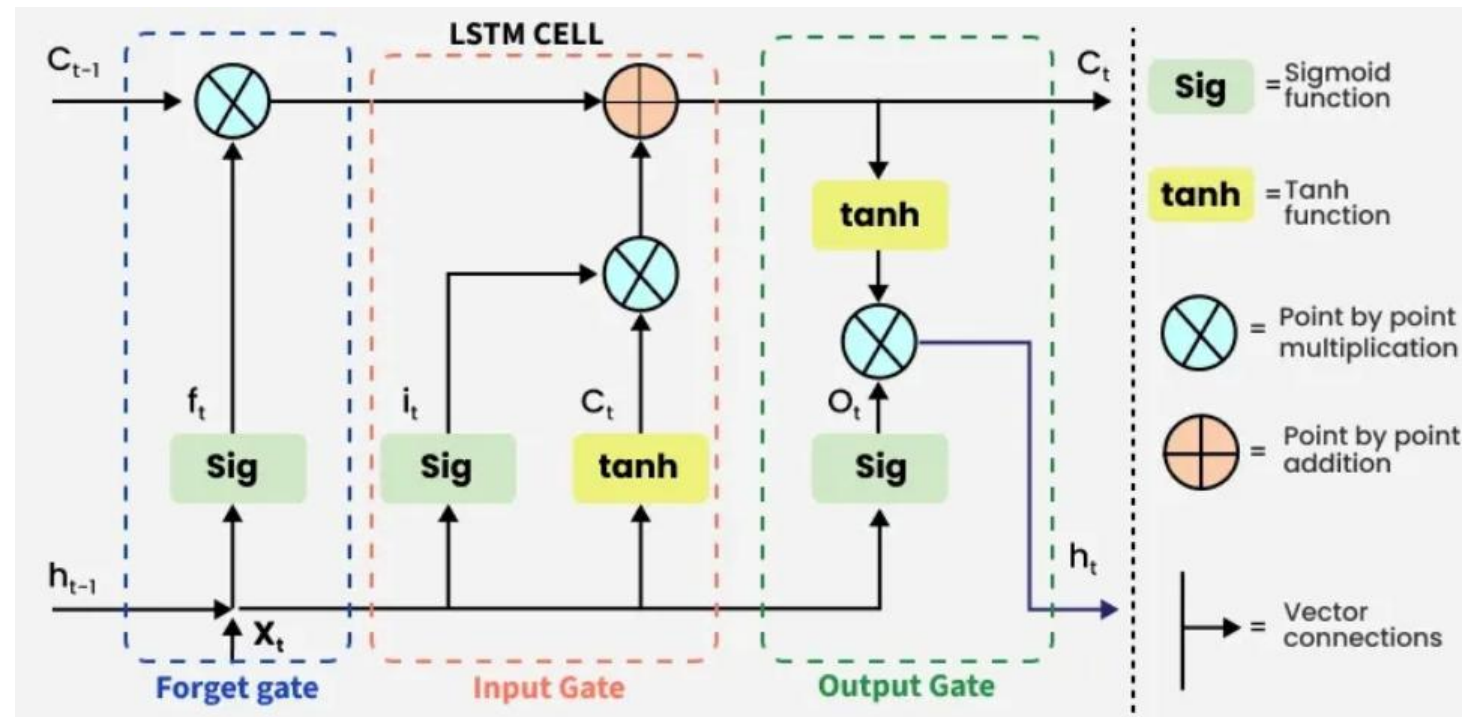
Long Short-Term Memory (LSTM) is an enhanced version of the Recurrent Neural Network (RNN) designed by Hochreiter and Schmidhuber. LSTMs can capture long-term dependencies in sequential data making them ideal for tasks like language translation, speech recognition and time series forecasting.

Unlike traditional RNNs which use a single hidden state passed through time LSTMs introduce a **memory cell** that holds information over extended periods addressing the challenge of learning long-term dependencies.

LSTM : Architecture

An LSTM has:

- **Cell state (C_t):** The "memory" highway
- **Hidden state (h_t):** The output/short-term memory
- **Three gates:**
 - **Input Gate:** Controls how much new information should be added to the cell state.
 - **Forget Gate:** Decides what past information should be discarded.
 - **Output Gate:** Regulates what information should be output at the current step. This selective memory enables LSTMs to handle long-term dependencies, making them ideal for tasks where earlier context is critical

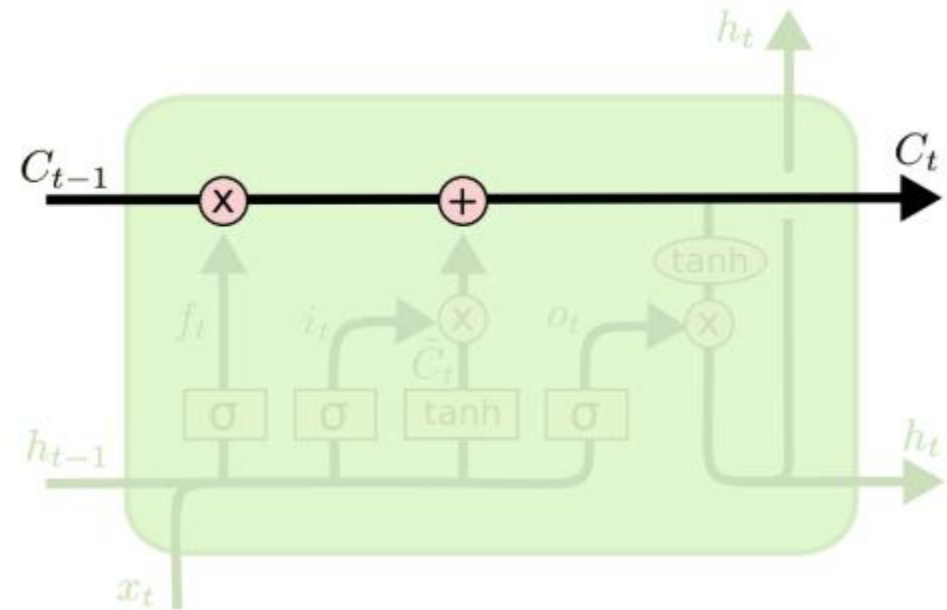


Step-by-Step LSTM Walk Through

The key to LSTMs is the cell state, the horizontal line running through the top of the diagram.

The cell state is kind of like a conveyor belt. It runs straight down the entire chain, with only some minor linear interactions. It's very easy for information to just flow along it unchanged.

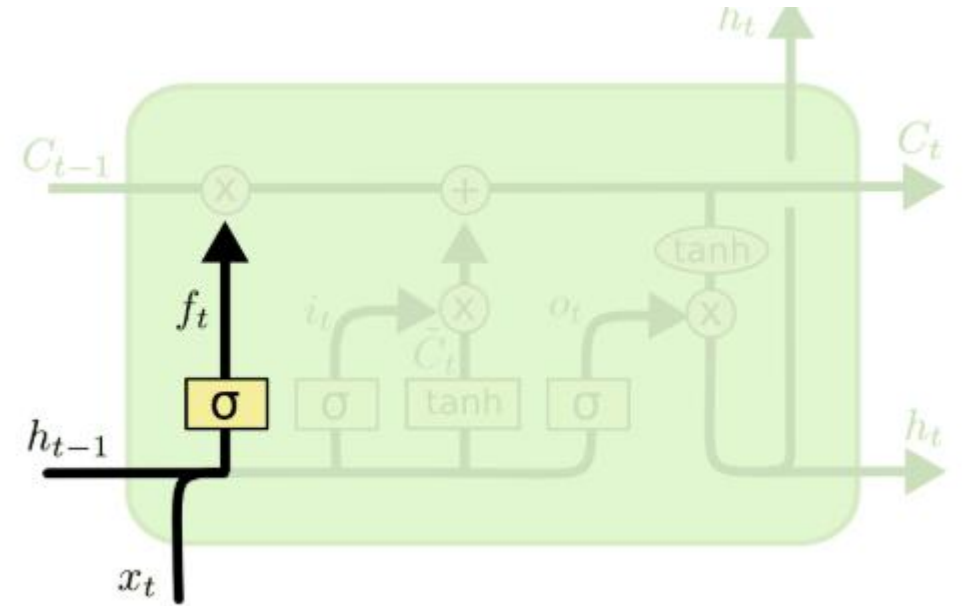
The LSTM does have the ability to remove or add information to the cell state, carefully regulated by structures called gates.



Step-by-Step LSTM Walk Through

The first step in our LSTM is to decide what information we're going to throw away from the cell state. This decision is made by a sigmoid layer called the "forget gate layer."

It looks at h_{t-1} and x_t , and outputs a number between 0 and 1 for each number in the cell state C_{t-1} . A 1 represents "completely keep this" while a 0 represents "completely get rid of this."

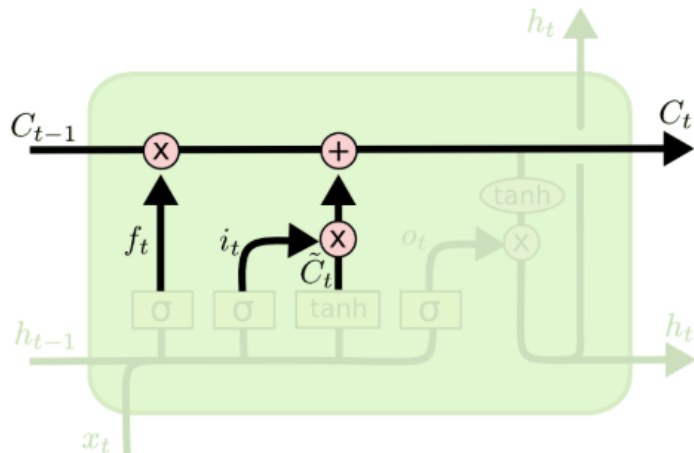


$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

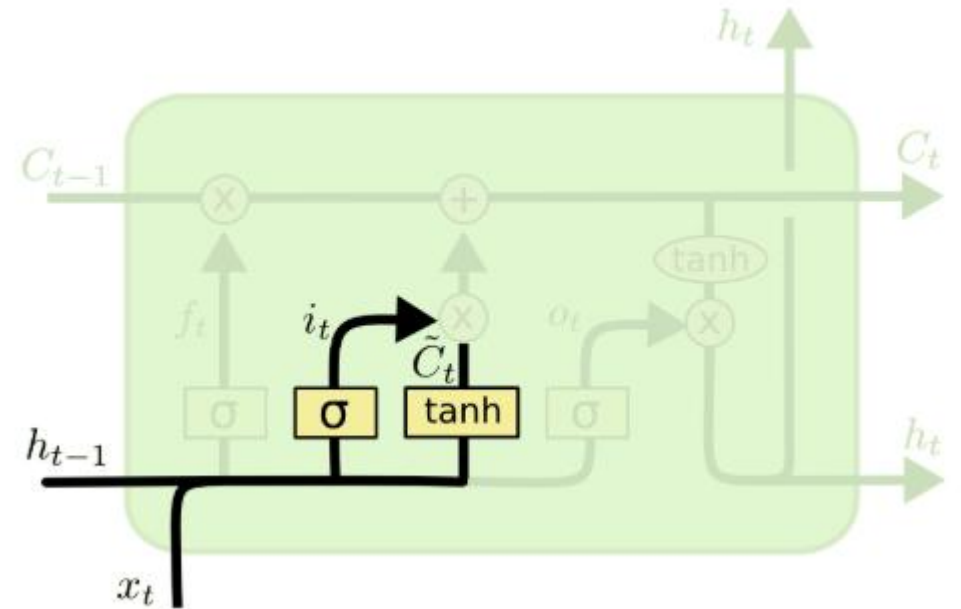
Step-by-Step LSTM Walk Through

The next step is to decide what new information we're going to store in the cell state. This has two parts. First, a sigmoid layer called the “input gate layer” decides which values we'll update. Next, a tanh layer creates a vector of new candidate values, $\tilde{C}_t$, that could be added to the state.

Finally, Update the old cell state C_{t-1} into the new cell state C_t :



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

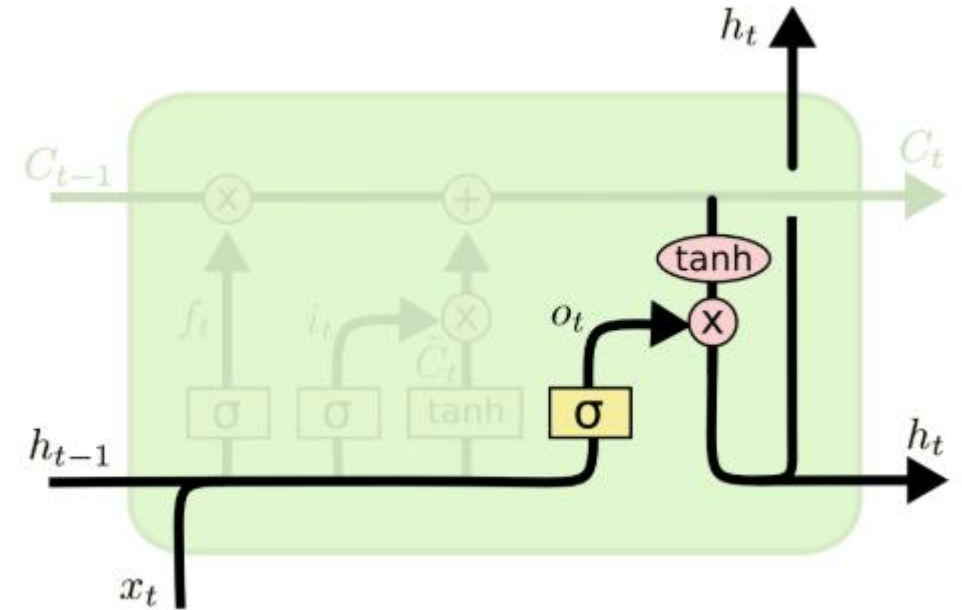


$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

Step-by-Step LSTM Walk Through

Finally, we need to decide what we're going to output. This output will be based on our cell state, but will be a filtered version. First, we run a sigmoid layer which decides what parts of the cell state we're going to output. Then, we put the cell state through tanh and multiply it by the output of the sigmoid gate, so that we only output the parts we decided to.



$$o_t = \sigma (W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh (C_t)$$

LSTM: Applications

Language Modeling: Used in tasks like language modeling, machine translation and text summarization. These networks learn the dependencies between words in a sentence to generate coherent and grammatically correct sentences.

Speech Recognition: Used in transcribing speech to text and recognizing spoken commands. By learning speech patterns they can match spoken words to corresponding text.

Time Series Forecasting: Used for predicting stock prices, weather and energy consumption. They learn patterns in time series data to predict future events.

Anomaly Detection: Used for detecting fraud or network intrusions. These networks can identify patterns in data that deviate drastically and flag them as potential anomalies.

Recommender Systems: In recommendation tasks like suggesting movies, music and books. They learn user behavior patterns to provide personalized suggestions.

Video Analysis: Applied in tasks such as object detection, activity recognition and action classification. When combined with Convolutional Neural Networks (CNNs) they help analyze video data and extract useful information.

LSTM: Applications

```
import torch.nn as nn
```

```
lstm = nn.LSTM(input_size=40, hidden_size=128, num_layers=2, batch_first=True)
```

- `input_size`: features per time step
- `hidden_size`: neurons in each LSTM layer
- `num_layers`: stacked LSTM layers

Gated Recurrent Units (GRU)

GRUs are a simplified version of LSTMs, they combine the input and forget gates into a single update gate helps in reducing the number of parameters and making the model less computationally demanding. Here's how it works:

Update Gate: This gate decides how much of the information from the past should be kept for future steps. It helps the GRU remember important details.

Reset Gate: This gate decides how much past information should be forgotten. If something is no longer relevant it forget that part.

